

1. Often in binary classification we are interested in the differences in the output of our current classifier, g , and an unknown function f that we are trying to learn. It is common in these cases to examine the quantity produced by $f(x)g(x)$ for a given input x . For this problem, let D be an arbitrary distribution on the domain $\{-1, 1\}^n$, and let $f, g: \{-1, 1\}^n \rightarrow \{-1, 1\}$ be two Boolean functions.

(a) [6 points] Prove that

$$\mathbb{E}_{x \sim D}[f(x) \neq g(x)] = \frac{1 - \mathbb{E}_{x \sim D}[f(x)g(x)]}{2}$$

- (b) [4 points] Would this still be true if the domain were some other domain (such as $\mathbb{R}^n$, where $\mathbb{R}$ denotes the real numbers, with say the Gaussian distribution) instead of $\{-1, 1\}^n$? If yes, justify your answer. If not, give a counterexample.
Note: Only the domain changes here. The output is still boolean.

1. (a) $\mathbb{P}_{x \sim D}[f(x) \neq g(x)] = \frac{1 - \mathbb{E}_{x \sim D}[f(x)g(x)]}{2}$

$$f, g: \{-1, 1\}^n \rightarrow \{-1, 1\}$$

$f(x) = g(x)$	$f(x)g(x)$
1	1
$f(x) \neq g(x)$	-1

$$\mathbb{E}_{x \sim D}[f(x)g(x)] = 1 \cdot \mathbb{P}_{x \sim D}[f(x) = g(x)] + (-1) \cdot \mathbb{P}_{x \sim D}[f(x) \neq g(x)] \quad \text{< def. of Expectation >}$$

$$= (1 - \mathbb{P}_{x \sim D}[f(x) \neq g(x)]) - \mathbb{P}_{x \sim D}[f(x) \neq g(x)] \quad \text{< Probability of complements >}$$

$$= 1 - 2 \mathbb{P}_{x \sim D}[f(x) \neq g(x)] \quad \text{algebra}$$

$$\Leftrightarrow \mathbb{P}_{x \sim D}[f(x) \neq g(x)] = \frac{1 - \mathbb{E}_{x \sim D}[f(x)g(x)]}{2} \quad \text{algebra}$$

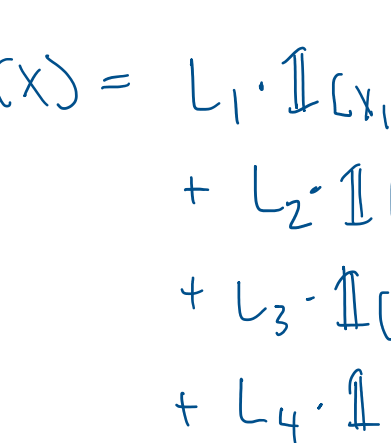
(b) Yes, the proof is agnostic to the domain.

It only depends on the boolean output.
No matter the support of X , $f(x) \neq g(x) \Rightarrow f(x)g(x) = -1$
and $f(x) = g(x) \Rightarrow f(x)g(x) = 1$

2. [10 points] Let f be a decision tree with t leaves over the variables $x = (x_1, \dots, x_n) \in \{-1, 1\}^n$. Explain how to write f as a multivariate polynomial $p(x_1, \dots, x_n)$ such that for every input $x \in \{-1, 1\}^n$, $f(x) = p(x)$. (You may interpret -1 as FALSE and 1 as TRUE or the other way round, at your preference.) (Hint: try to come up with an "indicator polynomial" for every leaf, i.e. one that evaluates to the leaf's value if x is such that that path is taken, and 0 otherwise.)

$$\text{Let } X = \{x_1, x_2, x_3\}$$

Assume our tree looks like this:



then polynomial representation would be:

$$p(x) = L_1 \cdot \mathbb{1}[x_1 = 1] \quad \text{(leaf)} \\ + L_2 \cdot \mathbb{1}[x_1 = -1] \cdot \mathbb{1}[x_2 = -1] \quad \text{(left, left)} \\ + L_3 \cdot \mathbb{1}[x_1 = -1] \cdot \mathbb{1}[x_2 = 1] \cdot \mathbb{1}[x_3 = -1] \quad \text{(left, right, left)} \\ + L_4 \cdot \mathbb{1}[x_1 = -1] \cdot \mathbb{1}[x_2 = 1] \cdot \mathbb{1}[x_3 = 1] \quad \text{(left, right, right)}$$

We can do this generally for tree $f(x)$ as defined.

Let $L_i, i \in \{1, 2, \dots, t\}$ = value of leaf i

Define an indicator I_i for each L_i indicating the path as above

$$\text{Then, } p(x) = \sum_{i=1}^t L_i \cdot I_i$$

3. [10 points] Compute a depth-two decision tree for the training data in table 1 using the Gini function, $C(a) = 2a(1-a)$ as described in class. What is the overall accuracy on the training data of the tree? For clarity, this will be a full binary tree and a full binary tree of depth-two has four leaves.

X	Y	Z	Number of positive examples	Number of negative examples
0	0	0	10	20
0	0	1	25	5
0	1	0	35	15
0	1	1	35	5
1	0	0	5	15
1	0	1	30	10
1	1	0	10	10
1	1	1	15	5

Table 1: decision tree training data

1. Root node:

$$\text{base: } C(\Pr[\text{NEG}]) \quad \text{total: 250} \quad \text{NEG: 35}$$

$$\text{case: } = 2 \cdot (85/165) \cdot (1 - 95/165) = 0.4995$$

$$X: \Pr[X=0]C(\Pr_{X=0}[\text{NEG}]) + \Pr[X=1]C(\Pr_{X=1}[\text{NEG}]) \quad X=0: 150 \quad \text{NEG}_{X=0}: 45 \\ = (150/250)C(45/150) + (100/250)C(40/100) \quad X=1: 100 \quad \text{NEG}_{X=1}: 40 \\ = 0.252 + 0.192 = 0.4440$$

$$Y: \Pr[Y=0]C(\Pr_{Y=0}[\text{NEG}]) + \Pr[Y=1]C(\Pr_{Y=1}[\text{NEG}]) \quad Y=0: 120 \quad \text{NEG}_{Y=0}: 50 \\ = (120/250)C(50/120) + (130/250)C(35/130) \quad Y=1: 130 \quad \text{NEG}_{Y=1}: 35 \\ = 0.2333 + 0.22046 = 0.45379$$

$$Z: \Pr[Z=0]C(\Pr_{Z=0}[\text{NEG}]) + \Pr[Z=1]C(\Pr_{Z=1}[\text{NEG}]) \quad Z=0: 120 \quad \text{NEG}_{Z=0}: 60 \\ = (120/250)C(60/120) + (130/250)C(25/130) \quad Z=1: 130 \quad \text{NEG}_{Z=1}: 25 \\ = 0.24 + 0.1615 = 0.4015$$

$$\text{Gain}(Z) = 0.4995 - 0.4015 = 0.0980 \rightarrow \text{split on } Z$$

2. Left node ($Z=0$):

$$\text{base: } C(\Pr_{Z=0}[\text{NEG}]) \quad Z=0: 120 \quad \text{NEG}_{Z=0}: 60$$

$$\text{case: } = 2 \cdot (60/120) \cdot (1 - 60/120) = 0.5000$$

$$X: \Pr_{Z=0}[X=0]C(\Pr_{X=0,Z=0}[\text{NEG}]) + \Pr_{Z=0}[X=1]C(\Pr_{X=1,Z=0}[\text{NEG}]) \quad X=0, Z=0: 80 \quad \text{NEG}_{X=0,Z=0}: 35 \\ = (80/120)C(35/80) + (40/120)C(25/40) \quad X=1, Z=0: 40 \quad \text{NEG}_{X=1,Z=0}: 25 \\ = 0.3281 + 0.1563 = 0.4844$$

$$Y: \Pr_{Z=0}[Y=0]C(\Pr_{Y=0,Z=0}[\text{NEG}]) + \Pr_{Z=0}[Y=1]C(\Pr_{Y=1,Z=0}[\text{NEG}]) \quad Y=0, Z=0: 50 \quad \text{NEG}_{Y=0,Z=0}: 35 \\ = (50/120)C(35/50) + (70/120)C(25/70) \quad Y=1, Z=0: 70 \quad \text{NEG}_{Y=1,Z=0}: 25 \\ = 0.175 + 0.2679 = 0.4429$$

$$\text{Gain}(Y) = 0.5000 - 0.4429 = 0.0571 \rightarrow \text{split on } Y$$

right node ($Z=1$):

$$\text{base: } C(\Pr_{Z=1}[\text{NEG}]) \quad Z=1: 130 \quad \text{NEG}_{Z=1}: 25$$

$$\text{case: } = 2 \cdot (25/130) \cdot (1 - 25/130) = 0.3107$$

$$X: \Pr_{Z=1}[X=0]C(\Pr_{X=0,Z=1}[\text{NEG}]) + \Pr_{Z=1}[X=1]C(\Pr_{X=1,Z=1}[\text{NEG}]) \quad X=0, Z=1: 70 \quad \text{NEG}_{X=0,Z=1}: 10 \\ = (70/130)C(35/70) + (60/130)C(15/60) \quad X=1, Z=1: 60 \quad \text{NEG}_{X=1,Z=1}: 15 \\ = 0.1319 + 0.1731 = 0.3049$$

$$Y: \Pr_{Z=1}[Y=0]C(\Pr_{Y=0,Z=1}[\text{NEG}]) + \Pr_{Z=1}[Y=1]C(\Pr_{Y=1,Z=1}[\text{NEG}]) \quad Y=0, Z=1: 70 \quad \text{NEG}_{Y=0,Z=1}: 15 \\ = (70/130)C(15/70) + (60/130)C(10/60) \quad Y=1, Z=1: 60 \quad \text{NEG}_{Y=1,Z=1}: 10 \\ = 0.0759 + 0.1282 = 0.1941$$

$$\text{Gain}(Y) = 0.3107 - 0.1941 = 0.1166 \rightarrow \text{split on } Y$$

3. Determine leaf values:

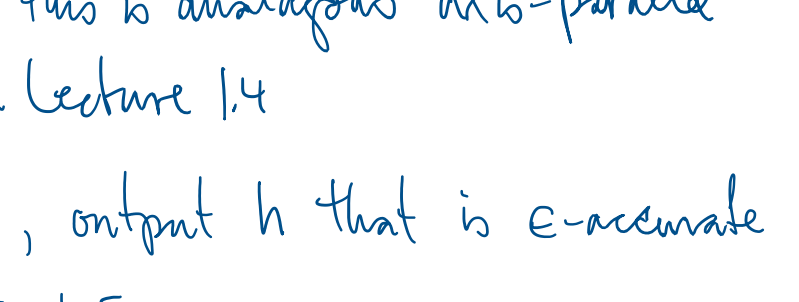
$$\text{majority}_{Z=0, Y=0}(\text{labels}) = 0 \quad (35 > 15)$$

$$\text{majority}_{Z=0, Y=1}(\text{labels}) = 1 \quad (45 > 25)$$

$$\text{majority}_{Z=1, Y=0}(\text{labels}) = 1 \quad (55 > 15)$$

$$\text{majority}_{Z=1, Y=1}(\text{labels}) = 1 \quad (50 > 10)$$

4. Accuracy



$$\text{correct: } 35 + 45 + 55 + 50 = 185$$

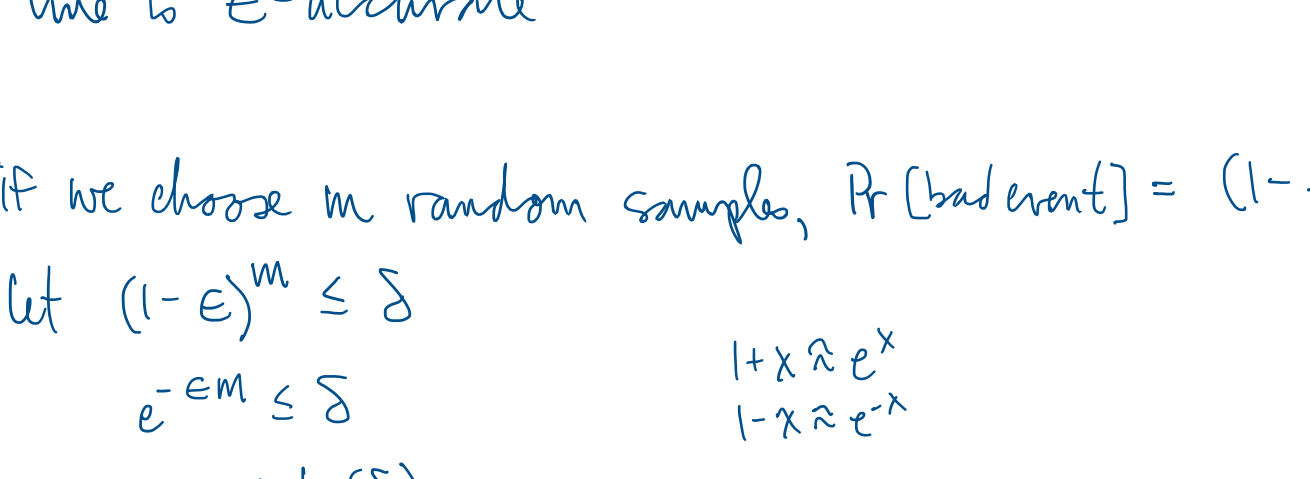
$$\text{accuracy: } 185/250 = 0.74$$

4. [10 points] Suppose the domain X is the real line, $\mathbb{R}$, and the labels lie in $Y = \{-1, 1\}$. Let C be the concept class consisting of simple threshold functions of the form h_θ for some $\theta \in \mathbb{R}$, where $h_\theta(x) = -1$ for all $x \leq \theta$ and $h_\theta(x) = 1$ otherwise. Give a simple and efficient PAC learning algorithm for C that uses only $m = O(\frac{1}{\epsilon} \log \frac{1}{\delta})$ training examples to output a classifier with error at most ϵ with probability at least $1 - \delta$.

* stuff suggested on Ed this is analogous axis-parallel rectangles problem in Lecture 1.4

goal: given ϵ, δ , output h that is ϵ -accurate

with $\Pr[S] \geq 1 - \delta$



- choose the θ nearest the left most positive point
- the only question remaining is how large does m need to be to achieve the goal
- we want to bound the probability of a "bad event"
- if the "bad event" does not occur, then tightest fitting line is ϵ -accurate

if we choose m random samples, $\Pr[\text{bad event}] = (1 - \epsilon)^m$

$$\text{let } (1 - \epsilon)^m \leq \delta$$

$$e^{-\epsilon m} \leq \delta \quad 1 + x \leq e^x$$

$$-\epsilon m \leq \ln(\delta)$$

$$m \geq \frac{\ln(\delta)}{-\epsilon} = \frac{1}{\epsilon} \ln\left(\frac{1}{\delta}\right)$$

$\Rightarrow$ choosing the tightest-fitting line w/ $m \geq O\left(\frac{1}{\epsilon} \ln\left(\frac{1}{\delta}\right)\right)$

samples is ϵ -accurate w/ prob $(1 - \delta)$

5. [6 points] In this problem we will show that the existence of an efficient mistake-bounded learner for a class C implies an efficient PAC learner for C .

Concretely, let C be a function class with domain $X \in \{-1, 1\}^n$ and binary labels $Y \in \{-1, 1\}$. Assume that C can be learned by algorithm A with some mistake bound t . You may assume you know the value t . You may also assume that at each iteration, A runs in time polynomial in n and that A only updates its state when it gets an example wrong. The concrete goal of this problem is to create a PAC-learning algorithm, B , that can PAC-learn concept class C with respect to an arbitrary distribution D over $\{-1, 1\}^n$ using algorithm A as a sub-routine.

In order to prove that learner B can PAC-learn concept class C , we must show that there exists a finite number of examples m , that we can draw from D such that B produces a hypothesis whose true error is more than ϵ with probability at most δ . First, fix some distribution D on X , and we will assume that the examples are labeled by an unknown $c \in C$. Additionally, for a hypothesis (i.e. function) $h: X \rightarrow Y$, let $\text{err}(h) = \mathbb{E}_{x \sim D}[h(x) \neq c(x)]$. Formally, we will need to bound m such that the following condition holds:

$$\forall \epsilon, \delta \in [0, 1], \exists m \in \mathbb{N} \quad \Pr_{x \sim D}[\text{err}(B(x_1^m)) > \epsilon] \leq \delta \quad x \sim D \quad (1)$$

where $B(x_1^m)$ denotes a hypothesis produced from D with m random draws of x from an arbitrary distribution D .

To find this m , we will first decompose it into blocks of examples of size k and make use of results based on a single block to find the bound necessary for m that satisfies condition 1.

Note: Using the identity $\Pr(\text{err}(h) > \epsilon) = \Pr(\text{err}(h) \leq \epsilon) = 1$, we can see that $\Pr(\text{err}(h) > \epsilon) \leq \delta$ is $\Pr(\text{err}(h) \leq 1 - \epsilon) \geq 1 - \delta$, which makes the connection to the definition of PAC-learning (described in lecture) explicit.

- (a) Fix a single arbitrary hypothesis $h': X \rightarrow Y$ produced by A and determine a lower bound on the number of examples, k , such that $\Pr(\text{err}(h') > \epsilon) \leq \delta'$. (The contrapositive view would be: with probability at least $1 - \delta'$, it must be the case that $\text{err}(h') \leq \epsilon$. Make sure this makes sense.)

$\text{err}(h')$ is the true error

assume $\text{err}(h') > \epsilon \Rightarrow \Pr(h' \text{ correctly describes } x) \leq (1 - \epsilon)$

$$\Rightarrow \Pr(h' \text{ and } \text{err}(h') > \epsilon) \leq (1 - \epsilon)^k$$

$$\text{bound } (1 - \epsilon)^k \leq \delta'$$

$$e^{-\epsilon k} \leq \delta'$$

$$-\epsilon k \leq \ln(\delta')$$

$$k \geq \frac{\ln(\frac{1}{\delta'})}{\epsilon}$$

to guarantee $\text{err}(h') \leq \epsilon$ w/ probability $> 1 - \delta'$

- (b) From part (a) we know that as long as a block is at least of size k , then if that block is classified correctly by a fixed arbitrary hypothesis h' we can effectively upper bound the probability of the "bad event" (i.e. A outputs h' s.t. $\text{err}(h') > \epsilon$) by δ' . However, our bound must apply to every h' that our algorithm B could output for an arbitrary distribution D over examples. With this in mind, how large should m be so that we can bound all hypotheses that could be output? (You may assume that algorithm B will know the mistake bound produced in the question.)

- A makes at most t mistakes
- Only updates when it makes a mistake
- A will return at most $t+1$ hypotheses

$$\text{need } m = (t+1)k = (t+1) \frac{\ln(\frac{1}{\delta'})}{\epsilon} \text{ samples}$$

- (c) Put everything together and fully describe (with proof) a PAC learner that is able to output a hypothesis with a true error at most ϵ with probability at least $1 - \delta$, given a mistake bounded learner A . To do this you should first describe your pseudocode for algorithm B which will use A as a sub-routine (no need for minute details or code, broad strokes will suffice). Then, prove there exists a finite number of m examples for B to PAC-learn C for all values of δ and ϵ by lower bounding m by a function of ϵ, δ , and t (i.e. finding a finite lower bound for m such that the PAC-learning requirements in 1 are satisfied).

- gather sample of size m
- pass sample of size $k < m$ to A
- if A succeeds on k examples, we're done because it has achieved its objective (true error can only be $> \epsilon$ with prob $\leq (1 - \delta)$ - to be proven)
- else go back to 2 with the next k samples
- ends if we make t mistakes

Proof:

$$1 \quad k \geq \frac{\ln(\frac{1}{\delta'})}{\epsilon} \Rightarrow \text{err}(h') > \epsilon \text{ w/ prob. } \leq \delta' \quad \text{(part a)}$$

$$2 \quad \text{let } (t+1)\delta' = \delta$$

$$\delta' = \frac{\delta}{t+1}$$

$$3 \Rightarrow k \geq \frac{\ln(\frac{1}{\delta'})}{\epsilon}$$

$$4 \quad \text{let } m = (t+1) \frac{\ln(\frac{1}{\delta'})}{\epsilon} \text{ samples}$$

5 if we pass $k = \frac{\ln(\frac{1}{\delta'})}{\epsilon}$ samples into A and are correct, this only would happen w/ prob $\frac{\delta}{t+1}$

6 Prob. (δ) happens w/ one of the $t+1$ hypotheses

$$\text{so } (t+1)\left(\frac{\delta}{t+1}\right) = \delta$$